

SentinelCore - Technical Review & Viva Q&A Guide

Stack: Spring Boot 3, Spring Security, JWT, JPA, React | Architecture: 3-Tier Layered

Section 1: Architecture & Package Quick Reference

Package / File	Purpose & Responsibility	Key Annotation / Pattern
BackendApplication.java	Main Application Bootstrap Entry Point	@SpringBootApplication
controller/	REST API Endpoints & Request Mapping	@RestController, @RequestMapping
service/	Core Business Logic, Rule & Alert Engines	@Service, @Transactional
repository/	Database CRUD Operations via Spring Data JPA	@Repository, JpaRepository
entity/	Database Table Schemas & Relational Mappings	@Entity, @Table, @Id
security/	JWT Filter, Token Provider & User Auth	OncePerRequestFilter, @Component
config/	Spring Security RBAC, CORS & Beans Config	@Configuration, @Bean
collector/	Real-time Server Log Monitoring & Syslog Parser	File Tailers, Regex Parsers
scheduler/	Automated Background Cron Tasks (Escalations)	@Scheduled(cron = '...')
exception/	Centralized Global Error & Exception Handling	@RestControllerAdvice
specification/	Dynamic Multi-Filter Database Queries	JpaSpecificationExecutor

Section 2: Top 20 Code-Level Viva Questions & Answers

Q1. Where is the main application entry point, and how does Spring Boot initialize?

Code Path: [backend/BackendApplication.java](#)

Answer: The application initializes from BackendApplication.java. The @SpringBootApplication annotation triggers auto-configuration, component scanning across all sub-packages, and initializes the Spring ApplicationContext while starting the embedded Tomcat container on port 8080.

Q2. Where are the Role-Based Access Control (RBAC) rules defined for Admin, Analyst, and Viewer?

Code Path: [backend/config/SecurityConfig.java \(lines 80-275\)](#)

Answer: All endpoint access rules are declared inside SecurityConfig.java. Using Spring Security matchers: VIEWER has read-only access (GET requests), ANALYST has read & write access (POST, PUT for operations), and ADMIN has full access including DELETE actions, User Management, and System Settings.

Q3. How do you store user passwords securely in the database?

Code Path: [backend/config/SecurityConfig.java](#) & [backend/service/AuthService.java](#)

Answer: We never store raw passwords. We register a BCryptPasswordEncoder bean in SecurityConfig.java. During user registration in AuthService.java, the raw password is automatically salted and hashed using the BCrypt hashing algorithm before being persisted to the database.

Q4. Where is the JWT token generated upon login, and what claims are stored inside it?

Code Path: [backend/security/JwtTokenProvider.java](#)

Answer: JWT generation is encapsulated inside JwtTokenProvider.java. Upon successful authentication, generateToken() builds a token containing user subject, roles/authorities claims, issue timestamp, and expiration time, cryptographically signed with HMAC-SHA256 using a secure secret key.

Q5. How does the backend intercept and validate incoming JWT tokens on every HTTP request?

Code Path: [backend/security/JwtAuthenticationFilter.java](#)

Answer: We implemented a custom `JwtAuthenticationFilter` extending `OncePerRequestFilter`. It intercepts each HTTP request, extracts the Bearer token from the Authorization header, verifies the signature using `JwtTokenProvider`, loads user authorities, and registers the Authentication object in `SecurityContextHolder`.

Q6. How does user signup work, and how do you prevent unauthorized Admin registrations?

Code Path: [backend/service/AuthService.java](#)

Answer: In `AuthService.java`, when a new user signs up, the service checks the requested role. To prevent regular users from creating elevated accounts, registering as an ADMIN or ANALYST strictly requires supplying a pre-configured secret passkey (Admin@only / Analyst@only).

Q7. How does the system seed default configurations like SMTP, escalation SLA, and retention limits?

Code Path: [backend/service/SettingsService.java \(lines 25-50\)](#)

Answer: In `SettingsService.java`, an `@PostConstruct` lifecycle method executes immediately after bean initialization. It checks if configuration keys like `SMTP_EMAIL`, `ALERT_ESCALATION_MINUTES`, and `LOG_RETENTION_DAYS` exist in the database; if not, it automatically seeds default key-value pairs.

Q8. Where is the core Alert Engine logic that evaluates events and triggers alerts?

Code Path: [backend/service/AlertEngineService.java](#)

Answer: The core detection engine is in `AlertEngineService.java`. It parses incoming security events and compares attributes against active rules from `AlertRuleRepository`. When criteria like event thresholds, failed login attempts, or threat signatures match, it automatically creates and saves a new Alert entity.

Q9. How are threat detection rules created and managed?

Code Path: [backend/service/AlertRuleService.java](#)

Answer: Detection rules specifying log source, condition type, threshold value, time window, and severity level are managed in `AlertRuleService.java`. The Alert Engine queries this service to evaluate live event streams against active rules.

Q10. What is a Playbook in SentinelCore, and how does automated incident response work?

Code Path: [backend/service/PlaybookService.java](#)

Answer: Playbooks provide SOAR (Security Orchestration, Automation, and Response) capabilities. In `PlaybookService.java`, predefined containment actions (such as blocking malicious IPs, isolating hosts, or dispatching emergency SOC alerts) execute automatically in sequential steps when a critical incident occurs.

Q11. How does an Alert escalate into a full Security Incident?

Code Path: [backend/service/IncidentService.java](#)

Answer: In `IncidentService.java`, high-severity or correlated alerts are promoted to an Incident. The service assigns a priority, links affected assets and source alerts, manages lifecycle states (OPEN -> INVESTIGATING -> MITIGATED -> CLOSED), and logs every modification to the audit trail.

Q12. How does IOC (Indicators of Compromise) matching and correlation work?

Code Path: [backend/service/IOCCorrelationService.java](#) & [IOCDetectionService.java](#)

Answer: The `IOCCorrelationService.java` cross-references incoming log IP addresses, domains, and file hashes with the threat intelligence database (ioc table). If a match is detected, it instantly elevates the threat score and tags the alert as an active compromise.

Q13. How does the system collect and parse external raw server logs?

Code Path: [backend/collector/LogWatcherService.java](#) & [LogParser.java](#)

Answer: The collector module continuously monitors server and auth log files. LogWatcherService.java tails log files in real-time, and LogParser.java tokenizes raw unstructured text into structured Java SecurityEvent objects for the Alert Engine.

Q14. How does the system handle background tasks like alert escalation or asset heartbeat monitoring?

Code Path: [backend/scheduler/AlertEscalationScheduler.java](#)

Answer: We utilize Spring's @Scheduled annotation. AlertEscalationScheduler.java periodically scans the database for unacknowledged critical alerts exceeding the SLA threshold (e.g. 10 minutes) and automatically promotes them to formal incidents.

Q15. How do you handle runtime errors and exceptions centrally across all REST APIs?

Code Path: [backend/controller/GlobalExceptionHandler.java](#)

Answer: We implemented GlobalExceptionHandler.java using @RestControllerAdvice. It intercepts runtime exceptions (such as ResourceNotFoundException or BadCredentialsException) across all controllers and returns structured JSON responses with proper HTTP status codes (400, 401, 404, 500).

Q16. How do you implement dynamic multi-criteria search without hardcoding SQL queries?

Code Path: [backend/specification/AssetSpecification.java](#)

Answer: We use Spring Data JPA Specification<T>. In AssetSpecification.java, SQL criteria predicates are constructed dynamically based on the presence of user filters (like IP address, OS type, or health status), generating optimal database queries on demand.

Q17. How is a database table mapped to a Java class in JPA?

Code Path: [backend/entity/Alert.java](#) (or [backend/entity/User.java](#))

Answer: In Alert.java, the @Entity annotation declares the class as a persistent model, and @Table(name = 'alerts') maps it to the database table. Fields are mapped with @Column, @Id defines the primary key, and @Enumerated(EnumType.STRING) persists enum states as readable strings.

Q18. How do Spring Data JPA Repositories execute database queries without manual SQL?

Code Path: [backend/repository/AlertRepository.java](#)

Answer: By extending JpaRepository<Alert, Long>, Spring Data JPA automatically provides CRUD operations (save, findById, findAll, deleteById). In addition, Spring generates query implementations automatically based on method naming conventions like findByStatusOrderByCreatedAtDesc.

Q19. How does a REST Controller receive HTTP requests and dispatch JSON responses?

Code Path: [backend/controller/AlertController.java](#)

Answer: In AlertController.java, the @RestController annotation ensures all method outputs serialize directly into JSON. @GetMapping maps incoming HTTP GET requests, calls the underlying AlertService, and returns the payload wrapped in a ResponseEntity with an HTTP 200 status code.

Q20. How does the React frontend attach the JWT token to outgoing API requests?

Code Path: [frontend/src/services/api.js](#) (lines 29-44)

Answer: In frontend/src/services/api.js, we configure an Axios request interceptor. Before any HTTP request leaves the browser, the interceptor fetches the JWT token from localStorage and automatically injects the Authorization: Bearer <token> header into the request.